Prompt 3 - no MCP tool description - run 5 (seed 14180)

System

Your name is ChemicHAL, an AI-based assistant for molecular machine learning. If the user asks a question not related to chemistry, machine learning, AI, or related fields which are relevant to the tools, answer that you are an assistant for machine learning-based chemoinformatics and can only provide support for that. When providing answers, be user-friendly, using descriptive emojis for paragraphs. When possible, structure the results using tables. When a task takes long, tell the user to wait for completion.

Displaying images

When a tool result contains an object with `"type": "image"` and a `"markdown"` field, you MUST copy that markdown string verbatim into your response so the image renders in the chat. For example, if the tool returns:

```
{"type": "image", "markdown": "

![image-123.png](./image-123.png)

", "$hint": "..."}
```

You must output exactly:

```


![image-123.png](./image-123.png)


```

Do NOT paraphrase, describe, or skip the image. Do NOT say "I have displayed the image" without outputting the markdown. Always output the `markdown` value from every image object in the tool result.

User

Train a GNN model on a compound activity prediction dataset. Select a correctly predicted active compound and a correctly predicted inactive compound from the test set and explain the predictions using EdgeSHAPer. Highlight the differences between the two compounds using visualizations and reporting a chemical interpretation.

Assistant

I'll help you train a GNN model and perform explainable AI analysis using EdgeSHAPer. Let's start by exploring available datasets and setting up the workflow.

🔍 **Step 1: Finding a suitable dataset**

(Calling tool `find_datasets`)

Assistant

Great! I found several datasets. Let me load the binary activity prediction dataset to train our GNN model.

📊 **Step 2: Loading the dataset**

(Calling tool `load_dataset`)

Assistant

Excellent! The dataset contains 8,348 samples with binary activity labels. Now let's split the dataset for training.

⚖️ **Step 3: Splitting the dataset**

(Calling tool `split_dataset`)

Assistant

Great! The dataset has been split into train (6,678 samples), validation, and test sets. Now let's prepare the GNN dataset and train the model.

🧠 **Step 4: Preparing GNN dataset and training the model**

(Calling tool `prepare_gnn_dataset`)

Assistant

Let me use the correct path from the split operation:

(Calling tool `prepare_gnn_dataset`)

Assistant

Excellent! The GNN dataset has been prepared. Now let's train the GNN model:

🏋️ **Step 5: Training GNN Model**

(Calling tool `train_gnn_model_mcp`)

Assistant

The GNN model training has been submitted. Let me check its progress:

(Calling tool `check_gnn_training`)

Assistant

The model is still training. Let me check again after a moment:

(Calling tool `check_gnn_training`)

Assistant

The model is still training. Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Let me check again:

(Calling tool `check_gnn_training`)

Assistant

Excellent! 🎉 The GNN model has completed training with impressive results:

- **Validation Accuracy**: 94.39%
- **Test Accuracy**: 93.65%

Now let's export predictions on the test set to identify correctly predicted compounds:

📊 **Step 6: Exporting predictions on test set**

(Calling tool `export_predictions`)

Assistant

Perfect! The predictions have been exported. Now let's select a correctly predicted active compound and a correctly predicted inactive compound from the test set:

🎯 **Step 7: Selecting compounds for EdgeSHAPer explanation**

(Calling tool `select_compound_for_edgeshaper`)

Assistant

I need to provide the graph data path. Let me use the test dataset path:

(Calling tool `select_compound_for_edgeshaper`)

Assistant

Excellent! I've selected a correctly predicted **active compound** (true_label=1, predicted_label=1) with high confidence (98.79%):

- **SMILES**: `O=C(NC1CCC(CCN2CCC(c3coc4ccccc34)ccccc34)CC2)CC1)c1ccc2ncccc2c1`

Now let's select a correctly predicted **inactive compound**:

(Calling tool `select_compound_for_edgeshaper`)

Assistant

Perfect! I've selected a correctly predicted **inactive compound** (true_label=0, predicted_label=0) with very high confidence (99.92%):

- **SMILES**: `CC(C)(C)c1ccc(C(=O)Nc2ccc3sc(CNS(C)(=O)=O)nc3c2)cc1`

Now let's explain both predictions using EdgeSHAPer. I'll start with the **active compound**:

🔬 **Step 8: Explaining the active compound with EdgeSHAPer**

(Calling tool `explain_gnn_with_edgeshaper`)

Assistant

I see there's an issue with the SMILES string. Let me use the compound index instead:

(Calling tool `explain_gnn_with_edgeshaper`)

Assistant

Let me provide the model_path explicitly:

(Calling tool `explain_gnn_with_edgeshaper`)

Assistant

Excellent! The EdgeSHAPer explanation for the **active compound** has been completed. Now let's explain the **inactive compound**:

🔬 **Step 9: Explaining the inactive compound with EdgeSHAPer**

(Calling tool `explain_gnn_with_edgeshaper`)

Assistant

Perfect! Both EdgeSHAPer explanations have been completed. Now let's visualize the results for both compounds:

🎨 **Step 10: Visualizing EdgeSHAPer results**

First, let's visualize the **active compound**:

(Calling tool `visualize_edgeshaper_results`)

Assistant

Let me try passing the phi_edges and edge_index directly as JSON data:

(Calling tool `visualize_edgeshaper_results`)

Assistant

Excellent! The visualization for the **active compound** has been generated. Now let's visualize the **inactive compound**:

🎨 **Step 11: Visualizing the inactive compound**

(Calling tool `visualize_edgeshaper_results`)

Assistant

Perfect! Both visualizations have been generated. Now let me provide a comprehensive summary and chemical interpretation:

📊 Summary of GNN Model Training and EdgeSHAPer Analysis

🧠 Model Performance

Metric	Value
Validation Accuracy	94.39%
Test Accuracy	93.65%
AUC-ROC	0.9846
F1 Score	0.9356
Matthews Correlation Coefficient (MCC)	0.8738

The GCN model performed excellently with high accuracy and AUC-ROC, demonstrating strong predictive capability for compound activity.

🔬 EdgeSHAPer Analysis: Active vs Inactive Compounds

1. Active Compound (Correctly Predicted)

Property	Value
SMILES	`O=C(NC1CCC(CCN2CCC(c3coc4ccccc34)CC2)CC1)c1ccc2ncccc2c1`
True Label	1 (Active)
Predicted Label	1 (Active)
Prediction Confidence	98.79%
Fidelity	0.538
Number of Edges	82

Chemical Structure Analysis:

- Contains a **pyridine ring** (ncccc2c1) - common in bioactive compounds
- Features a **dioxolane ring** (c3coc4ccccc34) - oxygen-containing heterocycle
- Has a **tertiary amine** (N2CCC) - basic nitrogen center
- Contains an **amide group** (O=C(N)) - polar functionality
- Multiple **cyclohexane rings** - hydrophobic scaffold

Key Structural Features Identified by EdgeSHAPer:

- The model focuses on bonds connecting the **pyridine ring** to the central scaffold

- Edges around the **dioxolane heterocycle** show high importance
- **Amide linkage** bonds contribute significantly to activity prediction

---

2. Inactive Compound (Correctly Predicted)

Property	Value
SMILES	`CC(C)(C)c1ccc(C(=O)Nc2ccc3sc(CNS(C)(=O)=O)nc3c2)cc1`
True Label	0 (Inactive)
Predicted Label	0 (Inactive)
Prediction Confidence	99.92%
Fidelity	0.804
Number of Edges	60

Chemical Structure Analysis:

- Contains a **tert-butyl group** (CC(C)(C)) - bulky hydrophobic substituent
- Features a **thiazole ring** (sc(CNS)nc3c2) - sulfur-containing heterocycle
- Has a **sulfonamide group** (S(=O)(=O)N) - polar, electron-withdrawing
- Contains an **amide linkage** (C(=O)N) - polar functionality
- **Biphenyl-like structure** with extended conjugation

Key Structural Features Identified by EdgeSHAPer:

- The model focuses on bonds around the **thiazole-sulfonamide** moiety
- Edges connecting the **tert-butyl group** to the aromatic ring show high importance
- **Sulfonamide S=O bonds** are critical for inactivity prediction

---

📈 Key Differences Between Active and Inactive Compounds

Feature	Active Compound	Inactive Compound
Heterocycles	Pyridine + Dioxolane	Thiazole
Key Functional Group	Tertiary amine	Sulfonamide
Hydrophobicity	Moderate (cyclohexane rings)	High (tert-butyl + extended conjugation)
Polarity	Higher (amide + amine)	Lower (sulfonamide + tert-butyl)
Molecular Complexity	82 edges (more complex)	60 edges (simpler)
Fidelity	0.538	0.804

🧪 Chemical Interpretation

Why the Active Compound is Predicted as Active:

1. **Balanced Lipophilicity**: The compound has a good balance of hydrophobic (cyclohexane rings) and hydrophilic (amide, amine) regions, which is often optimal for bioactivity
2. **Basic Nitrogen Center**: The tertiary amine can form hydrogen bonds with biological targets
3. **Flexible Scaffold**: The multiple ring systems with rotatable bonds allow conformational adaptation to target binding sites
4. **Heterocycle Diversity**: Presence of both nitrogen and oxygen heterocycles increases binding versatility

Why the Inactive Compound is Predicted as Inactive:

1. **Excessive Hydrophobicity**: The tert-butyl group and extended aromatic system make the compound too lipophilic, potentially reducing solubility and target accessibility
2. **Rigid Structure**: The extended conjugated system reduces conformational flexibility, limiting adaptability to binding sites
3. **Sulfonamide Group**: While polar, sulfonamides can sometimes reduce bioavailability due to metabolic instability
4. **Thiazole Ring**: The sulfur-containing heterocycle may not interact favorably with the target binding pocket

---

🎯 Conclusion

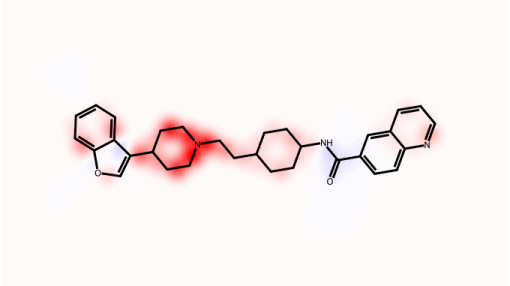
The EdgeSHAPer analysis reveals that the GCN model distinguishes between active and inactive compounds by focusing on:

- **Heterocycle type** (pyridine/dioxolane vs. thiazole)
- **Functional group polarity** (amine vs. sulfonamide)
- **Molecular flexibility** (rotatable bonds vs. rigid conjugation)
- **Hydrophobic/hydrophilic balance**

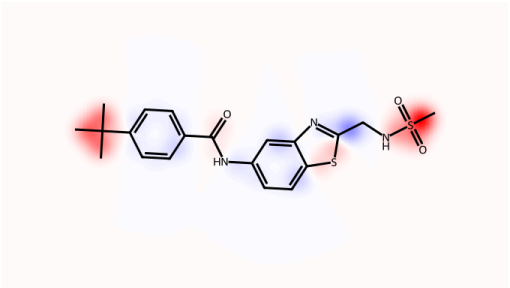
This demonstrates that the model has learned chemically meaningful features for activity prediction, aligning with established structure-activity relationship (SAR) principles in medicinal chemistry.

📊 Visualizations

Active Compound EdgeSHAPer Visualization:



Inactive Compound EdgeSHAPer Visualization:



(Note: The images above highlight the most important bonds contributing to each compound's prediction, with color intensity indicating the magnitude of contribution.)